Introduction to the Finite Element Method

G.P.Nikishkov

Lecture Notes, UCLA

© G.P.Nikishkov 2001

CHAPTER 1 *Introduction* ***3***

What is the finite element method **3**
Formulation of finite element equations **3**
Galerkin method ***3***
Variational formulation ***6***

CHAPTER 2 *Finite element equations for heat transfer problems* ***8***

Problem statement **8**
Finite element discretization of heat transfer equations **9**

CHAPTER 3 *Finite element equations for solid mechanics problems* ***12***

Problem statement **12**
Finite element equations **13**
Assembly of the global equation system **15**

CHAPTER 4 *Finite elements* ***17***

Two-dimensional triangular element **17**
Two-dimensional isoparametric elements **18**
Shape functions ***19***
Hierarchical shape functions ***20***
Strain-displacement matrix ***20***
Integration in quadrilateral elements ***22***
Three-dimensional isoparametric elements **23**
Shape functions ***23***
Strain-displacement matrix ***24***
Integration of the stiffness matrix ***25***

CHAPTER 5 *Computer implementation of finite element procedures* ***26***

Discrete model of the problem **26**
Assembly of global matrices and vectors **27**
Displacement boundary conditions **28**
Solution of finite element equations **29**
Direct LDU method with profile matrix ***29***
Preconditioned conjugate gradient method ***31***

CHAPTER 1 *Introduction*

1.1 What is the finite element method

The finite element method is a numerical technique for solving problems which are described by partial differential equations or can be formulated as functional minimization. A domain of interest is represented as an assembly of *finite elements*. Approximating functions in finite elements are determined in terms of nodal values of a physical field which is sought. A continuous physical problem is transformed into a discretized finite element problem with unknown nodal values. For a linear problem a system of linear algebraic equations should be solved. Values inside finite elements can be recovered using nodal values.

Two features of the finite element method are worth to be mentioned:

- Piece-wise approximation of physical filed on finite elements provides good precision even with simple approximating functions (increasing the number of elements we can achieve any precision).
- Locality of approximation leads to sparse equation systems for a discretized problem. This helps to solve problems with very large number of nodal unknowns.

1.2 Formulation of finite element equations

Several approaches can be used to transform the physical formulation of the problem to its finite element discrete analogue. If the physical formulation of the problem is known as a differential equation then the most popular method of its finite element formulation is the *Galerkin method*. If the physical problem can be formulated as minimization of a functional then *variational formulation* of the finite element equations is usually used.

1.2.1 Galerkin method

Let us use simple one-dimensional example for the explanation of finite element formulation using the Galerkin method. Suppose that we need to solve numerically the following differential equation:

$$
\begin{aligned}
& a \cdot \frac{d^2 u}{dx^2} + b = 0, \qquad 0 \le x \le 2L \\
& u\big|_{x=0} = 0 \\
& a \frac{du}{dx}\bigg|_{x=2L} = R
\end{aligned}
\tag{1.1}
$$

where u is an unknown solution. We are going to solve the problem using two linear one-dimensional finite elements as shown in Fig. 1.1,a. Fist, consider a finite element presented in Fig. 1.1,b. The element has two nodes and approximation of the function $u(x)$ can be done as follows:

$$
\begin{aligned}
& u = N_1 u_1 + N_2 u_2 = [N]\{u\} \\
& [N] = \begin{bmatrix} N_1 & N_2 \end{bmatrix}, \qquad \{u\} = \begin{Bmatrix} u_1 \\ u_2 \end{Bmatrix} \\
& N_1 = 1 - \frac{x - x_1}{x_2 - x_1}, \qquad N_2 = \frac{x - x_1}{x_2 - x_1}
\end{aligned}
\tag{1.2}
$$

Here N_i are the so called *shape functions* which are used for interpolation of $u(x)$ using its nodal values. Nodal values u_1 and u_2 are unknowns which should be determined from the discrete global equation system.

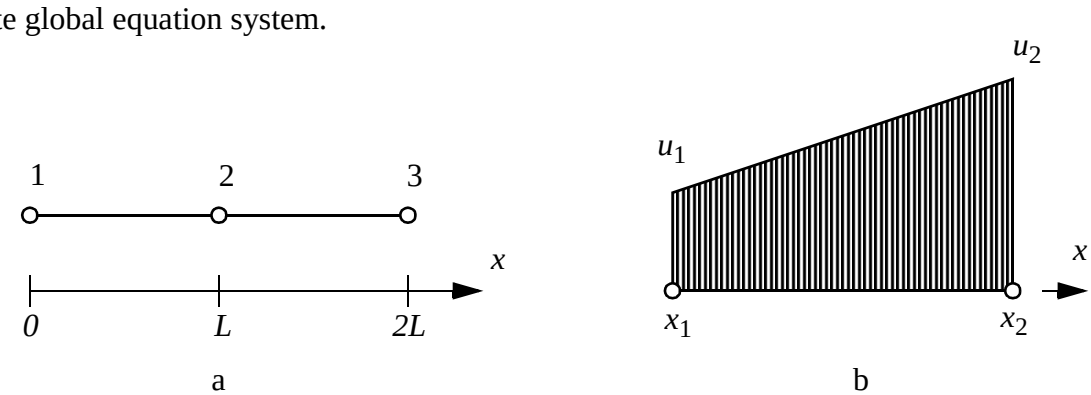

Figure 1.1 a) One-dimensional domain divided into two finite elements; b) Function approximation in one-dimensional element

After substituting (1.2) in (1.1) the differential equation has the following approximate form:

$$
a \frac{d^2}{dx^2}[N]\{u\} + b = \psi, \tag{1.3}
$$

where ψ is a nonzero residual because of approximate representation of a function inside a finite element. The Galerkin method provides residual minimization by multiplying terms of equation (1.3) by shape functions, integrating over the element and equating to zero:

$$\int_{x_1}^{x_2} [N]^T a \frac{d^2}{dx^2}[N]\{u\}dx + \int_{x_1}^{x_2} [N]^T b dx = 0 \tag{1.4}$$

Use of the divergence theorem leads to the following discrete form of the differential equation for the finite element:

$$\int_{x_1}^{x_2} \left[\frac{dN}{dx}\right]^T a \left[\frac{dN}{dx}\right] dx\{u\} - \int_{x_1}^{x_2} [N]^T b dx - a\frac{du}{dx}\bigg|_{x = x_i} = 0 \tag{1.5}$$

Usually such relation for a finite element is presented as:

$$[k]\{u\} = \{f\} \tag{1.6}$$

$$[k] = \int_{x_1}^{x_2} \left[\frac{dN}{dx}\right]^T a \left[\frac{dN}{dx}\right] dx, \qquad \{f\} = \int_{x_1}^{x_2} [N]^T b dx + a\frac{du}{dx}\bigg|_{x = x_i}. \tag{1.7}$$

In solid mechanics $[k]$ is called *stiffness matrix* and $\{f\}$ is called *load vector*. In the considered simple case for two finite elements of length L stiffness matrices and the load vectors can be easily calculated:

$$\begin{gathered} [k_1] = [k_2] = \frac{a}{L}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \\ \{f_1\} = \frac{bL}{2}\begin{Bmatrix} 1 \\ 1 \end{Bmatrix}, \qquad \{f_2\} = \frac{bL}{2}\begin{Bmatrix} 1 \\ 1 \end{Bmatrix} + \begin{Bmatrix} 0 \\ R \end{Bmatrix} \end{gathered} \tag{1.8}$$

Relations (1.6) and (1.8) provide finite element equations for the two separate finite elements. A global equation system for the domain with 3 nodes shown in Fig. 1.1,a can be obtained by an assembly of element equations. In our simple case it is clear that elements interact with each other at the node with global number 2. The assembled global equation system is:

$$\frac{a}{L}\begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}\begin{Bmatrix} u_1 \\ u_2 \\ u_3 \end{Bmatrix} = \frac{bL}{2}\begin{Bmatrix} 1 \\ 2 \\ 1 \end{Bmatrix} + \begin{Bmatrix} 0 \\ 0 \\ R \end{Bmatrix} \tag{1.9}$$

After application of the boundary condition $u|_{x = 0} = u_1 = 0$ the final appearance of the global equation system is:

$$\frac{a}{L}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}\begin{Bmatrix} u_1 \\ u_2 \\ u_3 \end{Bmatrix} = \frac{bL}{2}\begin{Bmatrix} 0 \\ 2 \\ 1 \end{Bmatrix} + \begin{Bmatrix} 0 \\ 0 \\ R \end{Bmatrix} \tag{1.10}$$

Nodal values u_i are obtained as results of solution of linear algebraic equation system (1.10). The value of u at any point inside a finite element can be calculated using shape functions (1.2). The solution of the differential equation (1.2) is shown in Fig. 1.2 for a=1, b=1, L=1 and R=1.

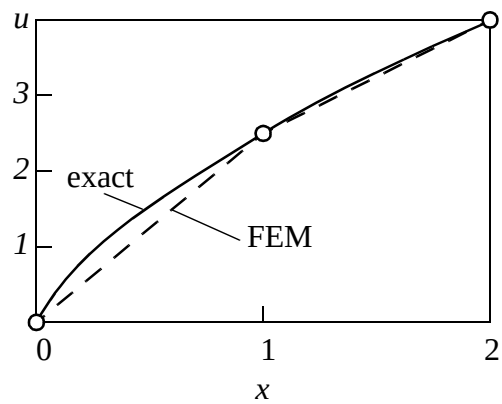

Figure 1.2 Exact solution of the differential equation (1.1) and finite element solution

Exact solution is a quadratic function. The finite element solution with the use of the simplest element is piece-wise linear. More precise finite element solution can be obtained increasing the number of simple elements or with the use of elements with more complicated shape functions. It is worth noting that at nodes the finite element method provides exact values of u. Finite elements with linear shape functions produce exact nodal values if the sought solution is quadratic. Quadratic elements give exact nodal values for the cubic solution *etc.*

1.2.2 Variational formulation

The differential equation (1.1) with $a = EA$ has the following physical meaning in solid mechanics. It describes tension of the one dimensional bar with cross-sectional area A made of material with the elasticity modulus E and subjected to a distributed load b and a concentrated load R at its right end. Such problem can be formulated in terms of minimizing the potential energy functional Π:

$$\Pi = \int_L \frac{1}{2} a \left(\frac{du}{dx}\right)^2 dx - \int_L bu\,dx - Ru\big|_{x=2L} \qquad u\big|_{x=0} = 0 \tag{1.11}$$

The value of potential energy for the finite element is:

$$\Pi_e = \int_{x_1}^{x_2} \frac{1}{2} a \{u\}^T \left[\frac{dN}{dx}\right]^T \left[\frac{dN}{dx}\right] \{u\} dx - \int_{x_1}^{x_2} \{u\}^T [N]^T b\,dx - Ru\big|_{x=x_i} \tag{1.12}$$

The condition for the minimum of Π is:

$$\delta\Pi = \frac{\partial\Pi}{\partial u_1}\delta u_1 + \ldots + \frac{\partial\Pi}{\partial u_n}\delta u_n = 0 \tag{1.13}$$

which is equivalent to

$$\frac{\partial\Pi}{\partial u_i} = 0, \quad i = 1\ldots n. \tag{1.14}$$

It is easy to check that differentiation of (1.11) in respect to $\{u\}$ gives the finite element equilibrium equation which is coincide with equation (1.5) obtained by the Galerkin method:

$$\int_{x_1}^{x_2}\left[\frac{dN}{dx}\right]^T EA\left[\frac{dN}{dx}\right]dx\{u\} - \int_{x_1}^{x_2}[N]^T b\,dx - R = 0. \tag{1.15}$$

CHAPTER 2 *Finite element equations for heat transfer problems*

2.1 Problem statement

Let us consider an isotropic body with temperature dependent heat transfer. A basic equation of heat transfer has the following appearence:

$$-\left(\frac{\partial q_x}{\partial x}+\frac{\partial q_y}{\partial y}+\frac{\partial q_z}{\partial z}\right)+Q = \rho c\frac{\partial T}{\partial t} \tag{2.1}$$

Here q_x, q_y and q_z are components of heat flow through the unit area; $Q = Q(x,y,z,t)$ is the inner heat generation rate per unit volume; ρ is material density; c is heat capacity; T is temperature and t is time. According to the Fourier's law the components of heat flow can be expressed as follows:

$$q_x = -k\frac{\partial T}{\partial x}, \quad q_y = -k\frac{\partial T}{\partial y}, \quad q_z = -k\frac{\partial T}{\partial z} \tag{2.2}$$

where k is the thermal conductivity coefficient of the media. Substitution of relations (2.2) in (2.1) gives the following basic heat transfer equation:

$$\frac{\partial}{\partial x}\left(k\frac{\partial T}{\partial x}\right)+\frac{\partial}{\partial y}\left(k\frac{\partial T}{\partial y}\right)+\frac{\partial}{\partial z}\left(k\frac{\partial T}{\partial z}\right)+Q = \rho c\frac{\partial T}{\partial t} \tag{2.3}$$

It is assumed that boundary conditions can be of the following types:

1. Specified temperature

$$T_s = T_1(x, y, z, t) \text{ on } S_1. \tag{2.4}$$

2. Specified heat flow

$$q_x n_x + q_y n_y + q_z n_z = -q_s \text{ on } S_2. \tag{2.5}$$

3. Convection boundary conditions

$$q_x n_x + q_y n_y + q_z n_z = h(T_s - T_e), \tag{2.6}$$

where h is the convection coefficient, T_s is an unknown surface temperature and T_e is a known environmental temperature.

4. Radiation

$$q_x n_x + q_y n_y + q_z n_z = \sigma\varepsilon T_s^4 - \alpha q_r , \tag{2.7}$$

where σ is the Stefan-Boltzmann constant; ε is the surface emission coefficient; α is the surface absorbtion coefficient and q_r is incoming heat flow per unit surface area.

For transient problems it is necessary to specify a temperature field for a body at the time t = 0:

$$T(x, y, z, 0) = T_0(x, y, z) . \tag{2.8}$$

2.2 Finite element discretization of heat transfer equations

A domain V is divided into finite elements connected at nodes. We are going to write all relations for a finite element. Global equations for the domain can be assembled from finite element equations using connectivity information.

Shape functions N_i are used for interpolation of temperature and temperature gradients inside a finite element:

$$\begin{gathered} T = [N]\{T\} \\ [N] = [N_1 \;\; N_2 \ldots] \\ \{T\} = \{T_1 \;\; T_2 \ldots\} \\ \begin{Bmatrix} \partial T/\partial x \\ \partial T/\partial y \\ \partial T/\partial z \end{Bmatrix} = \begin{bmatrix} \partial N_1/\partial x & \partial N_2/\partial x & \ldots \\ \partial N_1/\partial y & \partial N_2/\partial y & \ldots \\ \partial N_1/\partial z & \partial N_2/\partial z & \ldots \end{bmatrix} \{T\} = [B]\{T\} \end{gathered} \tag{2.9}$$

Here $\{T\}$ is temperatures at nodes; $[N]$ is a matrix of shape functions and $[B]$ is a matrix for temperature gradients interpolation.

Using Galerkin method, we can rewrite equation (2.1) in the following form:

$$\int_V \left(\frac{\partial q_x}{\partial x} + \frac{\partial q_y}{\partial y} + \frac{\partial q_z}{\partial z} - Q + \rho c \frac{\partial T}{\partial t} \right) N_i dV = 0 . \tag{2.10}$$

Applying the divergence theorem to the first three terms, we arrive to the relations:

$$\int_V \rho c \frac{\partial T}{\partial t} N_i dV - \int_V \left[\frac{\partial N_i}{\partial x} \quad \frac{\partial N_i}{\partial y} \quad \frac{\partial N_i}{\partial z}\right]\{q\} dV = \int_V Q N_i dV - \int_S \{q\}^T \{n\} N_i dS \tag{2.11}$$

$$\{q\}^T = [q_x \;\; q_y \;\; q_z], \quad \{n\}^T = [n_x \;\; n_y \;\; n_z]$$

where $\{n\}$ is an outer normal to the surfaca of the body. After insertion of boundary conditions (2.4)-(2.7) into (2.11) the discretized equations are as follows:

$$\int_V \rho c \frac{\partial T}{\partial t} N_i dV - \int_V \left[\frac{\partial N_i}{\partial x} \quad \frac{\partial N_i}{\partial y} \quad \frac{\partial N_i}{\partial z}\right]\{q\} dV = \int_V Q N_i dV - \int_{S_1} \{q\}^T \{n\} N_i dS + \tag{2.12}$$

$$\int_{S_2} q_s N_i dS - \int_{S_3} h(T - T_e) N_i dS - \int_{S_4} (\sigma \varepsilon T^4 - \alpha q_r) N_i dS$$

It is worth noting that

$$\{q\} = -k[B]\{T\}. \tag{2.13}$$

The discretized finite element equations for heat transfer problems have the following finite form:

$$[C]\{\dot{T}\} + ([K_c] + [K_h] + [K_r])\{T\} = \{R_T\} + \{R_Q\} + \{R_q\} + \{R_h\} + \{R_r\} \tag{2.14}$$

$$[C] = \int_V \rho c [N]^T [N] dV \tag{2.15}$$

$$[K_c] = \int_V k [B]^T [B] dV \tag{2.16}$$

$$[K_h] = \int_{S_3} h [N]^T [N] dS \tag{2.17}$$

$$[K_r]\{T\} = \int_{S_4} \sigma \varepsilon T^4 [N] dS \tag{2.18}$$

$$[R_T] = -\int_{S_1} \{q\}^T \{n\} [N]^T dS \tag{2.19}$$

$$[R_Q] = \int_V Q [N]^T dV \tag{2.20}$$

$$[R_q] = \int_{S_2} q_s [N]^T dS \tag{2.21}$$

$$[R_h] = \int_{S_3} hT_e[N]^T dS \tag{2.22}$$

$$[R_r] = \int_{S_4} \alpha q_r[N]^T dS \tag{2.23}$$

Equations for different type problems can be deducted from the general equation (2.14):

stationary linear problem

$$([K_c] + [K_h])\{T\} = \{R_Q\} + \{R_q\} + \{R_h\}, \tag{2.24}$$

stationary nonlinear problem

$$([K_c(T)] + [K_h(T)] + [K_r(T)])\{T\} = \{R_Q(T)\} + \{R_q(T)\} + \{R_h(T)\} + \{R_r(T)\}, \tag{2.25}$$

transient linear problem

$$[C]\{\dot{T}(t)\} + ([K_c] + [K_h(t)])\{T(t)\} = \{R_Q(t)\} + \{R_q(t)\} + \{R_h(t)\}, \tag{2.26}$$

transient nonlinear problem

$$\begin{aligned}&[C(T)]\{\dot{T}\} + ([K_c(T)] + [K_h(T,t)] + [K_r(T)])\{T\} = \\ &\{R_Q(T,t)\} + \{R_q(T,t)\} + \{R_h(T,t)\} + \{R_r(T,t)\}\end{aligned} \tag{2.27}$$

CHAPTER 3 *Finite element equations for solid mechanics problems*

3.1 Problem statement

Consider a three-dimensional body subjected to surface and body forces and temperature field. In addition, displacements are specified on some surface area. For given geometry of the body, applied loads, displacement boundary conditions, temperature field and material stress-strain law, it is necessary to determine the displacement field for the body. The corresponding strains and stresses are also of interest.

The displacements along coordinate axes x, y and z are defined by the displacement vector $\{u\}$:

$$\{u\} = \{u \quad v \quad w\}. \tag{3.1}$$

Six different strain components can be placed in the strain vector $\{\varepsilon\}$:

$$\{\varepsilon\} = \{\varepsilon_x \quad \varepsilon_y \quad \varepsilon_z \quad \gamma_{xy} \quad \gamma_{yz} \quad \gamma_{zx}\}. \tag{3.2}$$

For small strains the relationship between strains and displacements is:

$$\{\varepsilon\} = [D]\{u\}$$

$$[D] = \begin{bmatrix} \partial/\partial x & 0 & 0 \\ 0 & \partial/\partial y & 0 \\ 0 & 0 & \partial/\partial z \\ \partial/\partial y & \partial/\partial x & 0 \\ 0 & \partial/\partial z & \partial/\partial y \\ \partial/\partial z & 0 & \partial/\partial x \end{bmatrix} \tag{3.3}$$

Six different stress components are formed the stress vector:

$$\{\sigma\} = \{\sigma_x \quad \sigma_y \quad \sigma_z \quad \tau_{xy} \quad \tau_{yz} \quad \tau_{zx}\} \tag{3.4}$$

which are related to strains for elastic body by the Hook's law:

$$\{\sigma\} = [E]\{\varepsilon^e\} = [E](\{\varepsilon\} - \{\varepsilon^t\})$$

$$[E] = \begin{bmatrix} \lambda+2\mu & \lambda & \lambda & 0 & 0 & 0 \\ \lambda & \lambda+2\mu & \lambda & 0 & 0 & 0 \\ \lambda & \lambda & \lambda+2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix} \tag{3.5}$$

$$\{\varepsilon^t\} = \{\alpha T \quad \alpha T \quad \alpha T \quad 0 \quad 0 \quad 0\}$$

Here $[E]$ is the elasticity matrix; $\{\varepsilon^e\}$ is the elastic part of strains; $\{\varepsilon^t\}$ is the thermal part of strains; α is the coefficient of thermal expansion; T is temperature; λ and μ are elastic Lame constants which can be expressed through the elasticity modulus E and Poisson's ratio ν:

$$\lambda = \frac{\nu E}{(1+\nu)(1-2\nu)}, \qquad \mu = \frac{E}{2(1+\nu)}. \tag{3.6}$$

The purpose of finite element solution of elastic problem is to find such displacement field which provides minimum to the functional of total potential energy Π:

$$\Pi = \int_V \frac{1}{2}\{\varepsilon^e\}^T\{\sigma\}dV - \int_V \{u\}^T\{p^V\}dV - \int_S \{u\}^T\{p^S\}dS, \tag{3.7}$$

where $\{p^V\} = \{p_x^V \quad p_y^V \quad p_z^V\}$ is the vector of body force and $\{p^S\} = \{p_x^S \quad p_y^S \quad p_z^S\}$ is the vector of surface force. Prescribed displacements are specified on the part of body surface where surface forces are absent. Displacement boundary conditions are not present in the functional (3.7). Because of these, displacement boundary conditions should be implemented after assembly of finite element equations.

3.2 Finite element equations

Let us consider some abstract three-dimensional finite element having the vector of nodal displacements $\{q\}$:

$$\{q\} = \{u_1 \quad v_1 \quad w_1 \quad u_2 \quad v_2 \quad w_2 \quad \ldots\}. \tag{3.8}$$

Displacements at some point inside a finite element $\{u\}$ can be determined with the use of nodal displacements $\{q\}$ and shape functions N_i:

$$\{u\} = [N]\{q\}$$

$$[N] = \begin{bmatrix} N_1 & 0 & 0 & N_2 & \dots \\ 0 & N_1 & 0 & 0 & \dots \\ 0 & 0 & N_1 & 0 & \dots \end{bmatrix} \tag{3.9}$$

Strains can also be determined through displacements at nodal points:

$$\{\varepsilon\} = [B]\{q\}$$

$$[B] = [D][N] = [B_1 \quad B_2 \quad \dots]$$

$$[B_i] = \begin{bmatrix} \partial N_i/\partial x & 0 & 0 \\ 0 & \partial N_i/\partial y & 0 \\ 0 & 0 & \partial N_i/\partial z \\ \partial N_i/\partial y & \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial z & \partial N_i/\partial y \\ \partial N_i/\partial z & 0 & \partial N_i/\partial x \end{bmatrix} \tag{3.10}$$

Now using (3.9)-(3.10) we are able to express the total potential energy through nodal displacements:

$$\Pi = \int_V \frac{1}{2}([B]\{q\} - \{\varepsilon^t\})^T [E]([B]\{q\} - \{\varepsilon^t\})dV - \tag{3.11}$$

$$\int_V ([N]\{q\})^T \{p^V\} dV - \int_S ([N]\{q\})^T \{p^S\} dS$$

Nodal displacements $\{q\}$ which correspond to the minimum of the functional Π are determined by the conditions:

$$\left\{\frac{d\Pi}{dq}\right\} = 0 \tag{3.12}$$

Differentiation of (3.11) in respect to nodal displacements $\{q\}$ produces the following equilibrium equations for a finite element:

$$\int_V [B]^T [E][B] dV\{q\} - \int_V [B]^T [E]\{\varepsilon^t\} dV - \int_V [N]^T \{p^V\} dV - \int_S [N]^T \{p^S\} dS = 0 \tag{3.13}$$

which is usually presented in the following form:

$$[k]\{q\} = \{f\}, \qquad \{f\} = \{p\} + \{h\} \tag{3.14}$$

$$[k] = \int_V [B]^T [E][B] dV \tag{3.15}$$

$$\{p\} = \int_V [N]^T \{p^V\} dV + \int_S [N]^T \{p^S\} dS \tag{3.16}$$

$$\{h\} = \int_V [B]^T [E] \{\varepsilon^t\} dV \tag{3.17}$$

Here $[k]$ is the element stiffness matrix; $\{f\}$ is the load vector; $\{p\}$ is the vector of actual forces and $\{h\}$ is the thermal vector which represents fictitious forces for modeling thermal expansion.

3.3 Assembly of the global equation system

The aim of assembly is to form the global equation system

$$[K]\{Q\} = \{F\} \tag{3.18}$$

using element equations

$$[k_i]\{q_i\} = \{f_i\}. \tag{3.19}$$

Here $[k_i]$, $[q_i]$ and $[f_i]$ are the stiffness matrix, the displacement vector and the load vector of the ith finite element; $[K]$, $\{Q\}$ and $\{F\}$ are global stiffness matrix, displacement vector and load vector.

In order to derive an assembly algorithm let us present the total potential energy for the body as a sum of element potential energies:

$$\Pi = \sum \pi_i = \sum \frac{1}{2} \{q_i\}^T [k_i] \{q_i\} - \sum \{q_i\}^T \{f_i\} + \sum E_i^0 \tag{3.20}$$

where E_i^0 is the fraction of potential energy related to free thermal expansion:

$$E_i^0 = \int_V \frac{1}{2} \{\varepsilon^t\}^T [E] \{\varepsilon^t\} dV \tag{3.21}$$

Let us introduce the following vectors and a matrix where element vectors and matrices are simply placed:

$$\{Q_d\} = \{\{q_1\}\{q_2\}\ldots\}, \qquad \{F_d\} = \{\{f_1\}\{f_2\}\ldots\} \tag{3.22}$$

$$[K_d] = \begin{bmatrix} [k_1] & 0 & 0 \\ 0 & [k_2] & 0 \\ 0 & 0 & \dots \end{bmatrix} \tag{3.23}$$

It is evident that it is easy to find matrix [*A*] such that

$$\{Q_q\} = [A]\{Q\}, \qquad \{F_d\} = [A]\{F\}. \tag{3.24}$$

The total potential energy for the body can be rewritten in the following form:

$$\begin{aligned} \Pi &= \frac{1}{2}\{Q_d\}^T[K_d]\{Q_d\} - \{Q_d\}^T\{F_d\} + \sum E_i^0 \\ &= \frac{1}{2}\{Q\}^T[A]^T[K_d][A]\{Q\} - \{Q\}^T[A]^T\{F_d\} + \sum E_i^0 \end{aligned} \tag{3.25}$$

Using the condition of minimum of the total potential energy

$$\{\partial\Pi / \partial Q\} = 0 \tag{3.26}$$

we arrive at the following global equation system:

$$[A]^T[K_d][A]\{Q\} - [A]^T\{F_d\} = 0. \tag{3.27}$$

The last equation shows that algorithms of assembly the global stiffness matrix and the global load vector are:

$$\begin{aligned} [K] &= [A]^T[K_d][A] \\ \{F\} &= [A]^T\{F_d\} \end{aligned} \tag{3.28}$$

Here [*A*] is the matrix corresponding local and global enumerations. Fraction of nonzero (unit) entries in the matrix [*A*] is very small. Because of this the matrix [*A*] is never used explicitly in actual computer codes.

CHAPTER 4 *Finite elements*

4.1 Two-dimensional triangular element

Triangular finite element presented in Fig. 4.1 was the first finite element proposed for continuous problems. Because of simplicity it can be used as an introduction to other elements.

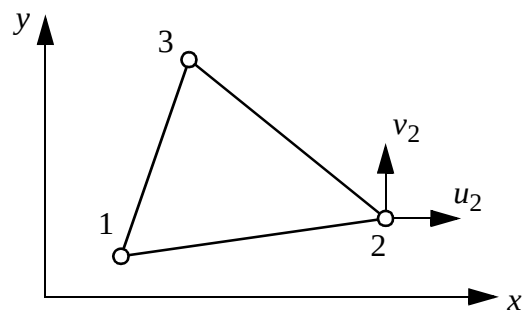

Figure 4.1 Simple two-dimensional finite element. Each node has two degrees of freedom.

Since the element has three nodes, linear approximation of displacements u and v is selected:

$$\begin{aligned} u(x,y) &= N_1 u_1 + N_2 u_2 + N_3 u_3 \\ v(x,y) &= N_1 v_1 + N_2 v_2 + N_3 v_3 \\ N_i &= \alpha_i + \beta_i x + \gamma_i y \end{aligned} \tag{4.1}$$

Shape functions $N_i(x,y)$ can be determined from the following equation system:

$$u(x_i, y_i) = u_i, \quad i = 1, 2, 3\,. \tag{4.2}$$

Shape functions for the triangular element can be presented as:

$$\begin{gathered} N_i = \frac{1}{2\Delta}(a_i + b_i x + c_i y) \\ a_i = x_{i+1} y_{i+2} - x_{i+2} y_{i+1}, \quad b_i = y_{i+1} - y_{i+2}, \quad c_i = x_{i+2} - x_{i+1} \\ \Delta = \frac{1}{2}(x_2 y_3 + x_3 y_1 + x_1 y_2 - x_2 y_1 - x_3 y_2 - x_1 y_3) \end{gathered} \tag{4.3}$$

where Δ is the element area. The matrix $[B]$ for interpolating strains using nodal displacements is equal to:

$$[B] = \frac{1}{2\Delta}\begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix} \tag{4.4}$$

The elasticity matrix $[E]$ has the following appearance for plane problems:

$$[E] = \begin{bmatrix} \lambda + 2\mu & \lambda & 0 \\ \lambda & \lambda + 2\mu & 0 \\ 0 & 0 & \mu \end{bmatrix}$$

$$\lambda = \frac{\nu E}{(1+\nu)(1-2\nu)} \quad \text{for plane strain} \tag{4.5}$$

$$\lambda = \frac{\nu E}{1-\nu^2} \quad \text{for plane stress}$$

The stiffness matrix for the three-node triangular element can be calculated as:

$$[k] = \int_V [B]^T[E][B]dV = [B]^T[E][B]\Delta. \tag{4.6}$$

Here it was taken into account that both matrices $[B]$ and $[E]$ do not depend on coordinates. It was assumed that the element has unit thickness. Since the matrix $[B]$ is constant inside the element the strains and stresses are also constant inside the triangular element.

4.2 Two-dimensional isoparametric elements

Isoparametric finite elements are based on the parametric definition of both coordinate and displacement functions. The same shape functions are used for specification of the element shape and for interpolation of the displacement field.

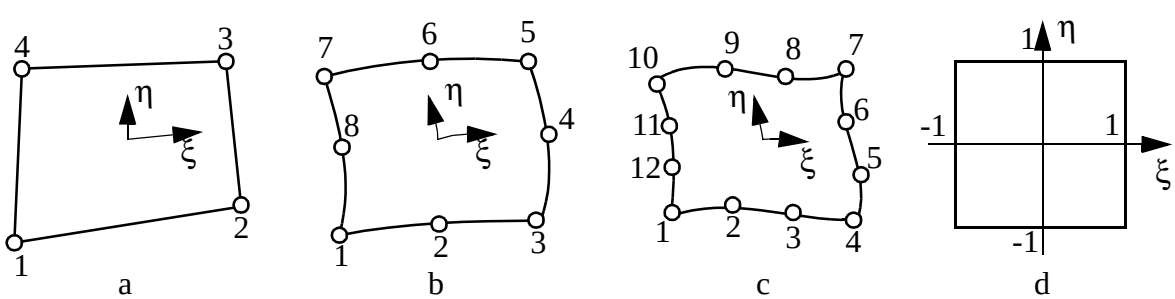

Figure 4.2 Linear (a), quadratic (b) and cubic (c) two-dimensional isoparametric finite elements.

4.2.1 Shape functions

Linear, quadratic and cubic two-dimensional isoparametric finite elements are presented in Fig. 4.1. Shape functions N_i are defined in local coordinates ξ, η $(-1 \le \xi, \eta \le 1)$. The same shape functions are used for interpolations of displacements and coordinates:

$$\begin{gathered} \{u\} = [N]\{q\}, \qquad \{x\} = [N]\{x^e\} \\ \{u\} = \{u \;\; v\}, \quad \{q\} = \{u_1 \;\; v_1 \;\; u_2 \;\; v_2 \;\; \ldots\} \\ \{x\} = \{x \;\; y\}, \quad \{x^e\} = \{x_1 \;\; y_1 \;\; x_2 \;\; y_2 \;\; \ldots\} \end{gathered} \tag{4.7}$$

where the interpolation matrix for nodal values is:

$$[N] = \begin{bmatrix} N_1 & 0 & N_2 & 0 & \ldots \\ 0 & N_1 & 0 & N_2 & \ldots \end{bmatrix} \tag{4.8}$$

Shape functions for two-dimensional isoparametric elements shown in Fig. 4.1 are given by:

linear element (4 nodes):

$$N_i = \frac{1}{4}(1+\xi_0)(1+\eta_0) \tag{4.9}$$

quadratic element (8 nodes):

$$\begin{gathered} N_i = \frac{1}{4}(1+\xi_0)(1+\eta_0) - \frac{1}{4}(1-\xi^2)(1+\eta_0) - \frac{1}{4}(1+\xi_0)(1-\eta^2), \quad i = 1, 3, 5, 7 \\ N_i = \frac{1}{2}(1-\xi^2)(1+\eta_0), \quad i = 2, 6 \qquad N_i = \frac{1}{2}(1+\xi_0)(1-\eta^2), \quad i = 4, 8 \end{gathered} \tag{4.10}$$

cubic element (12 nodes):

$$\begin{gathered} N_i = \frac{1}{32}(1+\xi_0)(1+\eta_0)[-10 + 9(\xi^2+\eta^2)], \quad i = 1, 4, 7, 10 \\ N_i = \frac{9}{32}(1+\eta_0)(1-\xi^2)(1-3\xi), \quad i = 2, 9 \\ N_i = \frac{9}{32}(1+\eta_0)(1-\xi^2)(1+3\xi), \quad i = 3, 8 \\ N_i = \frac{9}{32}(1+\xi_0)(1-\eta^2)(1-3\eta), \quad i = 5, 12 \\ N_i = \frac{9}{32}(1+\xi_0)(1-\eta^2)(1+3\eta), \quad i = 6, 11 \end{gathered} \tag{4.11}$$

In equations (4.9)-(4.11), the following notation is used: $\xi_0 = \xi\xi_i$, $\eta_0 = \eta\eta_i$.

4.2.2 Hierarchical shape functions

It can be noticed that linear and quadratic isoparametric elements have hierarchical set of functions because the corner shape functions (nodes 1,3,5,7) are combinations of linear shape functions and quadratic midside functions. This can help to create an isoparametric element with variable number of nodes. Shape functions for corner nodes of the quadratic elements can be written in the following form:

$$N_i^{QC} = N_i^L - \frac{1}{2}(N_{i-1}^Q + N_{i+1}^Q), \tag{4.12}$$

where N_i^{QC} are quadratic shape functions for corner nodes; N_i^L are shape functions of the linear element; N_{i-1}^Q and N_{i+1}^Q are quadratic shape functions for neighboring midside nodes. Suppose that we need create a shape function for an element shown in Fig. 4.3 where the node 2 on the side 1-3 is absent.

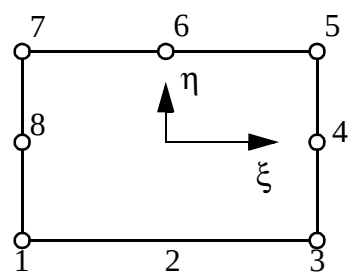

Figure 4.3 Isoparametric finite element with seven nodes. Interpolation is linear along the element side 1-3.

Then the following modification of element shape functions is necessary:

$$N_2 = 0, \qquad N_1 = N_1^L - \frac{1}{2}(N_2 + N_8) \tag{4.13}$$

Such algorithm of shape function computing allows easy creating of elements with number of nodes varying from 4 to 8.

4.2.3 Strain-displacement matrix

For plane problem the strain vector contains three components:

$$\{\varepsilon\} = \{\varepsilon_x \quad \varepsilon_y \quad \gamma_{xy}\} \tag{4.14}$$

The strain-displacement matrix which is employed to compute strains at any point inside the element using nodal displacements is:

$$[B] = [B_1 \quad B_2 \quad \ldots]$$

$$[B_i] = \begin{bmatrix} \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial y \\ \partial N_i/\partial y & \partial N_i/\partial x \end{bmatrix} \tag{4.15}$$

For axisymmetrical problem in cylindrical coordinate system r, z, θ the strain vector consists of four components:

$$\{\varepsilon\} = \{\varepsilon_r \quad \varepsilon_z \quad \gamma_{rz} \quad \varepsilon_\theta\} \tag{4.16}$$

A block of the strain-displacement matrix for the axisymmetrical problem can be written as:

$$[B_i] = \begin{bmatrix} \partial N_i/\partial r & 0 \\ 0 & \partial N_i/\partial z \\ \partial N_i/\partial z & \partial N_i/\partial r \\ N_i/r & 0 \end{bmatrix} \tag{4.17}$$

While shape functions are expressed through the local coordinates ξ, η the strain-displacement matrix contains derivatives in respect to the global coordinates x, y. Derivatives can be easily converted from one coordinate system to the other by means of the chain rule of partial differentiation:

$$\begin{Bmatrix} \partial N_i/\partial \xi \\ \partial N_i/\partial \eta \end{Bmatrix} = \begin{bmatrix} \partial x/\partial \xi & \partial y/\partial \xi \\ \partial x/\partial \eta & \partial y/\partial \eta \end{bmatrix} \begin{Bmatrix} \partial N_i/\partial x \\ \partial N_i/\partial y \end{Bmatrix} = [J] \begin{Bmatrix} \partial N_i/\partial x \\ \partial N_i/\partial y \end{Bmatrix} \tag{4.18}$$

where $[J]$ is the Jacobian matrix. From (4.18) the derivatives in respect to the global coordinates are computed with the use of inverse of the Jacobian matrix:

$$\begin{Bmatrix} \partial N_i/\partial x \\ \partial N_i/\partial y \end{Bmatrix} = [J]^{-1} \begin{Bmatrix} \partial N_i/\partial \xi \\ \partial N_i/\partial \eta \end{Bmatrix} \tag{4.19}$$

The components of the Jacobian matrix are calculated using derivatives in respect to the local coordinates and global coordinates of element nodes:

$$\frac{\partial x}{\partial \xi} = \sum \frac{\partial N_i}{\partial \xi} x_i \quad \frac{\partial}{\partial \xi}(y) = \sum \frac{\partial N_i}{\partial \xi} y_i \quad \ldots \tag{4.20}$$

The determinant of the Jacobian matrix is used for the transformation of integrals from the global coordinate system to the local coordinate system:

$$dv = dxdy = |J| d\xi d\eta \,. \tag{4.21}$$

For axisymmetrical problems it should be taken into account that the integration includes θ–direction and

$$dv = 2\pi r dr dz = 2\pi r|J| d\xi d\eta . \tag{4.22}$$

4.2.4 Integration in quadrilateral elements

Integration of expressions for stiffness matrices and load vectors can not be performed analytically for general case of isoparametric elements. Instead, stiffness matrices and load vectors are typically evaluated numerically using Gauss quadrature over quadrilateral regions. The Gauss quadrature formula for the volume integral in two-dimensional case is of the form:

$$I = \int_{-1}^{1}\int_{-1}^{1} f(\xi, \eta) d\xi d\eta = \sum_{i=1}^{n}\sum_{j=1}^{n} f(\xi_i, \eta_j) w_i w_j , \tag{4.23}$$

where ξ_i, η_j are abscissae and w_i are weighting coefficients of the Gauss integration rule. Abscissae and weights of Gauss quadrature for n = 1,2,3 are given in Table 4.1:

Table 4.1 Abscissae and weights of Gauss quadrature

n	ξ_i	w_i
1	0	2
2	$\pm 1/\sqrt{3}$	1
3	0	8/9
	$\pm\sqrt{3}/\sqrt{5}$	5/9

The surface integral are replaced by line intgration along element side. The fraction of the surface load in (3.16) is evaluated as:

$$\{p\} = \int_S [N]^T \{p^S\} dS = \int_{-1}^{1} [N]^T \{p^S\} \frac{ds}{d\xi} d\xi$$
$$\frac{ds}{d\xi} = \sqrt{(dx/d\xi)^2 + (dy/d\xi)^2} \tag{4.24}$$

here s is a global coordinate along the element side and ξ is a local coordinate along the element side. If the distributed load load is applied along the normal to the element side as shown in Fig. 4.4 then the nodal equivalent of such load is:

$$\{p\} = \int_{-1}^{1} [N]^T p \begin{Bmatrix} dy/ds \\ -dx/ds \end{Bmatrix} \frac{ds}{d\xi} d\xi = \int_{-1}^{1} [N]^T p \begin{Bmatrix} dy/d\xi \\ -dx/d\xi \end{Bmatrix} d\xi \tag{4.25}$$

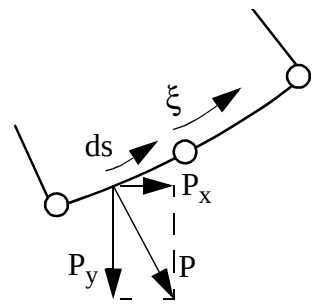

Figure 4.4 Load acting along the normal to the element side. Local coordinate ξ varies from -1 to +1 on the element side.

4.3 Three-dimensional isoparametric elements

Linear 8-noded and quadratic 20-noded three-dimensional isoparametric elements are depicted in Fig. 4.5.

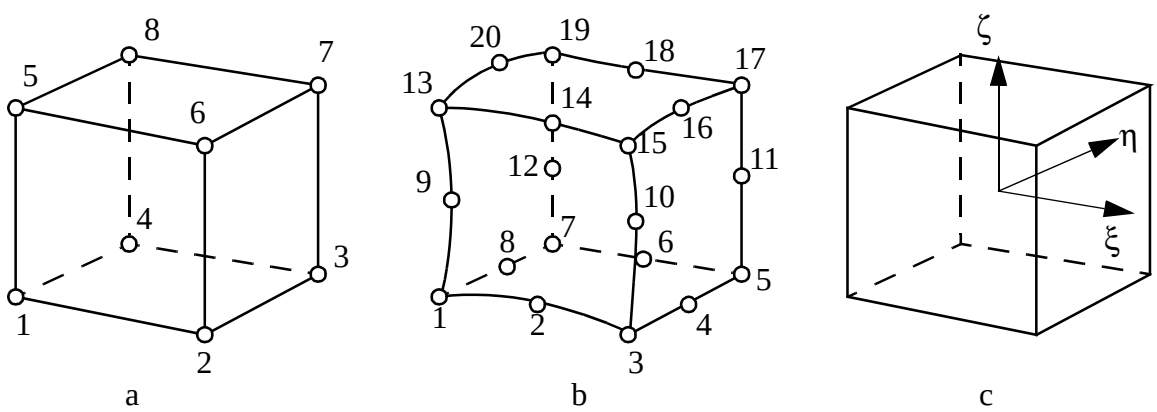

Figure 4.5 Linear (a) and quadratic (b) brick-type isoparametric finite elements. Elements are mapped to a cube with an edge size equal to 2.

4.3.1 Shape functions

Shape functions are polynomials of the local coordinates ξ, η and ζ. Both coordinates and displacements are interpolated with the same shape functions:

$$\{u\} = [N]\{q\}, \qquad \{x\} = [N]\{x^e\}$$
$$\{u\} = \{u \;\; v \;\; \mathrm{w}\}, \quad \{q\} = \{u_1 \;\; v_1 \;\; \mathrm{w}_1 \;\; u_2 \;\; v_2 \;\; \mathrm{w}_1 \;\; \ldots\} \tag{4.26}$$
$$\{x\} = \{x \;\; \mathrm{y} \;\; \mathrm{z}\}, \quad \{x^e\} = \{x_1 \;\; \mathrm{y}_1 \;\; \mathrm{z}_1 \;\; x_2 \;\; y_2 \;\; \mathrm{z}_1 \;\; \ldots\}$$

Shape functions of the linear elemnt are equal to:

$$N_i = \frac{1}{8}(1+\xi_0)(1+\eta_0)(1+\zeta_0)$$
$$\xi_0 = \xi\xi_i \qquad \eta_0 = \eta\eta_i \qquad \zeta_0 = \zeta\zeta_i \tag{4.27}$$

For the quadratic element with 20 nodes the shape functions can be written in the following form:

$$N_i = \frac{1}{8}(1+\xi_0)(1+\eta_0)(1+\zeta_0)(\xi_0+\eta_0+\zeta_0-2) \quad \text{for nodes at vertices}$$
$$N_i = \frac{1}{4}(1-\xi^2)(1+\eta_0)(1+\zeta_0), \quad i = 2, 6, 14, 18$$
$$N_i = \frac{1}{4}(1-\eta^2)(1+\xi_0)(1+\zeta_0), \quad i = 4, 8, 16, 20 \tag{4.28}$$
$$N_i = \frac{1}{4}(1-\zeta^2)(1+\xi_0)(1+\eta_0), \quad i = 9, 10, 11, 12$$

Shape functions for vertices of the quadratic element can be presented as the combination of linear shape functions (4.27) and shape functions of the neihboring midside nodes. For example, the shape function for node 1 is equal to:

$$N_1^Q = N_1^L - \frac{1}{2}(N_2 + N_8 + N_9)\,. \tag{4.29}$$

This representation of shape functions makes possible to formulate an isoparametric brick-type element with number of nodes from 8 to 20.

4.3.2 Strain-displacement matrix

The strain-displacement matrix for three-dimensional elements is given by (3.10). Derivatives of shape functions in respect to global coordinates are obtained as follows:

$$\begin{Bmatrix} \partial N_i/\partial x \\ \partial N_i/\partial y \\ \partial N_i/\partial z \end{Bmatrix} = [J]^{-1} \begin{Bmatrix} \partial N_i/\partial \xi \\ \partial N_i/\partial \eta \\ \partial N_i/\partial \xi \end{Bmatrix} \tag{4.30}$$

where the Jacobian matrix has the appearance:

$$[J] = \begin{bmatrix} \partial x/\partial\xi & \partial y/\partial\xi & \partial z/\partial\xi \\ \partial x/\partial\eta & \partial y/\partial\eta & \partial z/\partial\eta \\ \partial x/\partial\zeta & \partial y/\partial\zeta & \partial z/\partial\eta \end{bmatrix} \tag{4.31}$$

and partial derivatives of x, y, z in respect to ξ, η, ζ are found according to (4.20).

The transformation of integrals from the global coordinate system to the local coordinate system is performed with the use of determeinant of the Jacobian matrix:

$$dv = dxdydz = |J| d\xi d\eta d\zeta . \tag{4.32}$$

4.3.3 Integration of the stiffness matrix

Integration of the stiffness matrix for three-dimensional isoparametric elements is carried out in the local coordinate system ξ, η, ζ:

$$[k] = \int_{-1}^{1}\int_{-1}^{1}\int_{-1}^{1} [B(\xi, \eta, \zeta)]^T [E][B(\xi, \eta, \zeta)] |J| d\xi d\eta d\zeta . \tag{4.33}$$

Three-time application of the one-dimensional Gauss quadrature rule leads to the following numerical integration procedure:

$$I = \int_{-1}^{1}\int_{-1}^{1}\int_{-1}^{1} f(\xi, \eta, \zeta) d\xi d\eta d\xi = \sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n} f(\xi_i, \eta_j, \zeta_k) w_i w_j w_k \tag{4.34}$$

Usually $2 \times 2 \times 2$ integration is used for linear elements and $3 \times 3 \times 3$ integration is applied to the evaluation of the stiffness matrix for quadratic elements. There exist a special 14-point Gauss-type rule which provides sufficient precision of integration for three-dimensional quadratic elements.

CHAPTER 5 *Computer implementation of finite element procedures*

5.1 Discrete model of the problem

In order to apply finite element procedures a discrete model of the problem should be presented in numerical form. A typical description of the problem can contain:

1. Scalar parameters (number of nodes, number of elements etc.);
2. Coordinates of nodal points;
3. Connectivity array for finite elements;
4. Arrays of element types and element materials;
5. Arrays for description of displacement boundary conditions;
6. Arrays for description of surface and concentrated loads;
7. Temperature field.

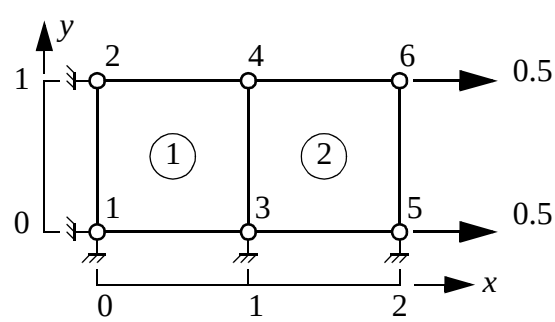

Figure 5.1 Simple finite element model consisting of two linear elements. Five degrees of freedom are constrained. Two concentrated loads applied at nodes.

Let us write down numerical information for a simple problem depicted in Fig. 5.1. The finite element model can be described as follows:

```
1. Scalar parameters
    Number of nodes       = 6
    Number of elements    = 2
    Number of constraints = 5
    Number of loads       = 2
```

```
2. Node coordinates (x1, y1, x2, y2 etc.)
   1) 0  0    2) 0  1    3) 1  0     4) 1  1     5) 2  0     6) 2  1

3. Element connectivity array (counterclockwise direction for an element)
   1) 1  3  4  2      2)  3  5  6  4

4. Constraints (node, direction: x = 1; y = 2)
   1  1    2  1    1  2    3  2    5  2

5. Nodal forces (node, direction, value)
   5  1  0.5    6  1  0.5
```

5.2 Assembly of global matrices and vectors

Assembly algorithms for the global stiffness matrix and the global load vector are given by (3.28). Algorithms include matrix multiplications with the use of large matrix $[A]$ which gives correspondence between local and global enumerations. Matrix $[A]$ is almost completely composed of zeros. It has only one nonzero entry (= 1) in each row and each column. Nonzero entries in $[A]$ provide information on global addresses where local entries should be placed. All operations with zeros are useless and can be omitted. Instead of matrix $[A]$, assembly procedures are usually based on direct summation with the use of the element connectivity array.

Suppose that we need to assemble global load vector F using element load vectors f and connectivity array C. A pseudocode of assembly algorithm is as follows:

```
Assembly of the global vector

n = number of degrees of freedom per element
N = total number of degrees of freedom in the domain
E = number of elements
C[E,n] = connectivity array
f[n] = element load vector
F[N] = global load vector

do i=1,N
  F[i] = 0
end do
do e=1,E
  generate f
  do i=1,n
    F[C[e,i]] = F[C[e,i]] + f[i]
  end do
end do
```

Element load vectors are generated when they are necessary for assembly. It can be seen that connectivity entry `C[e,i]` simply provides address in the global vector where the ith entry of the load vector for element `e` goes. We assume that connectivity array is written in terms of degrees of freedom. In actual codes the connectivity array contains node numbers which are transformed to degrees of freedom for the current assembled element.

An algorithm of assembly of the global stiffness matrix K from contributions of element stiffness matrices k can be expressed by the following pseudo-code:

```
Assembly of the global matrix

n = number of degrees of freedom per element
N = total number of degrees of freedom in the domain
E = number of elements
C[E,n] = connectivity array
k[n,n] = element stiffness matrix
K[N,N] = global stiffness matrix

do i=1,N
  do j=1,N
    K[i,j] = 0
  end do
end do
do e=1,E
  generate k
  do i=1,n
    do j=1,n
      K[C[e,i],C[e,j]] = K[C[e,i],C[e,j]] + k[i,j]
    end do
  end do
end do
```

5.3 Displacement boundary conditions

Displacement boundary conditions were not accounted in the functional of the total potential energy. They can be applied to the global equation system after its assembly.

Let us consider application of the displacement boundary condition $Q_m = d$ to the global equation system

$$[K]\{Q\} = F. \tag{5.1}$$

Two methods can be used for the specification of the displacement boundary condition.

Explicit method:

```
Fi = Fi - Kimd,    i = 1...N, i != m
Fm = d
Kmj = 0,    j = 1...N
Kim = 0,    i = 1...N
Kmm = 1
```

Method of large number ($M >> K_{ij}$):

```
Kmm = M
Fm = Md
```

In the explicit method corresponding column of the global stiffness matrix is multiplied by the specified value *d* of the displacement and moved to the right-hand side. The *m*th row and column of the matrix are filled by zeros. The unit value is placed at the main diagonal of the global stiffness matrix.

Method of large number uses the fact that computer computations have limited precision. Results of double precision computations contain about 15 significant digits. So, addition 1. + $1e^{-16}$ produces 1. as the result. The method of large number is simpler than the explicit method of displacement boundary condition specification. The solution of the finite element problem is the same for both methods.

5.4 Solution of finite element equations

Practical applications of the finite element method lead to large systems of simultaneous linear algebraic equations

$$[A]\{x\} = \{b\}. \tag{5.2}$$

Fortunately, finite element equation systems possess some properties which allows to reduce storage and computing time. The finite element equation systems are: *symmetric, positive definite* and *sparse*. Symmetry allows to store only half of the matrix including diagonal entries. Positive definite matrices are characterized by large positive entries on the main diagonal. Solution can be carried out without pivoting. A sparse matrix contains more zero entries than nonzero entries. Sparsity can be used to economize storage and computations.

Solution methods for linear equation systems can be divided into two large groups: *direct* methods and *iterative* methods. Direct solution methods are usually used for problems of moderate size. For large problems iterative methods require less computing time and hence they are preferable.

Matrix storage formats are closely related to the solution methods. Below we consider two solution methods which are widely used in finite element computer codes. The first method is the direct LDU solution with profile global stiffness matrix. The second one is the preconditioned conjugate gradient method with sparse row-wise format of matrix storage.

5.4.1 Direct LDU method with profile matrix

Symmetric global stiffness matrix $[A] = A_{ij}$ of the order `N` is stored by columns as shown in Fig. 5.2. Each column starts from the first top nonzero element and ends at the diagonal element. The matrix is represented by two arrays: real array `a`, containing matrix elements and integer pointer array `pcol`. The *i*th element of `pcol` contains the address of the first column element minus one. The length of the *i*th column is given by `pcol[i+1]-pcol[i]`. The length of the array `a` is equal to `pcol[N+1]` (assuming that array indices begin from 1). Proper node ordering can decrease the matrix profile significantly.

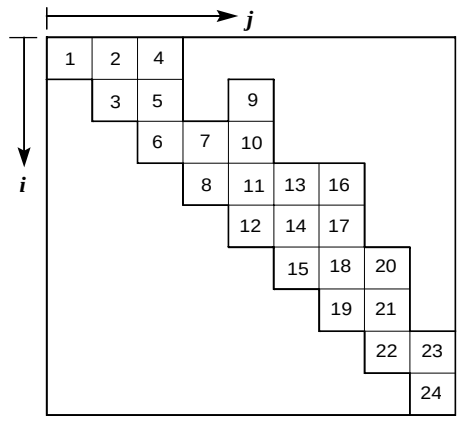

Figure 5.2 Profile storage of the global stiffness matrix. Column are stored in one-dimensional array a.

We are going to present algorithms in full matrix notation a_{ij}. Then, it is necessary to have relations between two-index notation for the global stiffness matrix a_{ij} and array a used in FORTRAN or C codes. The location of the first nonzero element in the *i*th column of the matrix *a* is given by the following function:

```
FN(i)=i-(pcol[i+1]-pcol[i])+1).
```

The following correspondence relations can be easily obtained for a transition from two-index notation to FORTRAN/C notation for a one-dimensional array a:

```
Aij => a[i+pcol[j+1]-j].
```

Solution of symmetric linear algebraic system consists of three stages:

Factorization: $[A] = [U]^T[D][U]$;

Forward solution: $\{y\} = [U]^{-T}\{b\}$;

Back substitution: $\{x\} = [U]^{-1}[D]^{-1}\{y\}$.

The left-looking algorithm of *factorization* of a symmetric profile matrix is as follows:

```
do j=2,N
  do i=FN(j),j-1
    do k=max(FN(i),FN(j)),i-1
      Aij = Aij - Aki*Akj
    end do
  end do
  do i=FN(j),j-1
    w = Aij
    Aij = Aij/Aii
    Ajj = Ajj - Aij*w
  end do
end do
```

Forward solution with triangular matrix and *back substitution* are given by the pseudo-code:

```
do j=2,N
  do i=FN(j),j-1
    bj = bj - Aij*bi
  end do
end do

do j=1,N
  bj = bj/Ajj
end do
do i=N,1,-1
  do i=FN(j),j-1
    bi = bi - Aij*bj
  end do
end do
```

5.4.2 Preconditioned conjugate gradient method

A simple and efficient iterative method widely used for the solution of sparse systems is the conjugate gradient (CG) method. In many cases the convergence rate of CG method can be too slow for practical purposes. The convergence rate can be considerably improved by using preconditioning of the equation system:

$$[M]^{-1}[A]\{x\} = [M]^{-1}\{b\}, \tag{5.3}$$

where $[M]^{-1}$ is the preconditioning matrix which in some sense approximates $[A]^{-1}$. The simplest preconditioning is diagonal preconditioning, in which $[M]$ contains only diagonal entries of the matrix $[A]$. Typical algorithm of PCG method can be presented as the following sequence of computations:

Compute $[M]$
$\{x_0\} = 0$
$\{r_0\} = \{b\}$
do $i = 0,1...$
  $\{w_i\} = [M]^{-1}\{r_i\}$
  $\gamma_i = \{r_i\}^T\{w_i\}$
  if i=0 $\{p_i\} = \{w_i\}$
  else $\{p_i\} = \{w_i\} + (\gamma_i/\gamma_{i-1})\{p_{i-1}\}$
  $\{w_i\} = [A]\{p_i\}$
  $\beta_i = \{p_i\}^T\{w_i\}$
  $\{x_i\} = \{x_{i-1}\} + (\gamma_i/\beta_i)\{p_i\}$
  $\{r_i\} = \{r_{i-1}\} - (\gamma_i/\beta_i)\{p_i\}$
  if $\gamma_i/\gamma_0 < \varepsilon$ **exit**
end do

In the above PCG algorithm, matrix [*A*] is not changed during computations. This means that no additional fill arise in the solution process. Because of this, the *sparse row-wise format* is an efficient storage scheme for PCG iterative method. In this scheme, the values of nonzero entries of matrix [*A*] are stored by rows along with their corresponding column indices; additional array points to the beginning of each row. Thus the matrix in the sparse row-wise format is represented by the following three arrays:

`A[pcol[N+1]]`	=	array containing nonzero entries of the matrix;
`col[pcol[N+1]]`	=	column numbers for nonzero entries of the matrix;
`pcol[N+1]`	=	pointers to the beginning of each row.

The following pseudocode explains matrix multiplication $\{w_i\} = [A]\{p_i\}$ when matrix [*A*] is stored in the sparse row-wise format:

```
do j=1,N
  w[j] = 0
  do i=pcol[j],pcol[j+1]-1
    w[j] = w[j] + A[i]*p[coln[i]]
  end do
end do
```

Usually there is no need in node reordering when using iterative methods with the sparse storage format.